

ESP32 MQTT fire alarm lab guide

Build the Week 9 prototype, practise the Week 8 MQTT demo, and collect evidence that your ESP32, broker and MQTT Explorer communicate in both directions. The default alarm triggers above 30 C, escalates after 2 seconds, and resets at or below 28 C. Local and remote acknowledgement silence the current alarm.

This guide matches the supplied project and the attached Week 8 MQTT slides, printed slides 32-37 (PDF pages 1-6). It assumes the existing uPesy ESP32 WROVER board setting, a 10 k Ω NTC thermistor with B = 3950 K, and a small two-pin passive piezo. A starter-kit label alone does not confirm the thermistor or buzzer specification. This is a classroom simulation, not a life-safety fire detector.

What you need

Item	Quantity and purpose
ESP32, USB data cable, breadboard and jumpers	One set; power the ESP32 from USB
10 k Ω NTC thermistor	One; senses temperature; no polarity
10 k Ω fixed resistor	One; forms the thermistor voltage divider
Two-pin passive piezo buzzer	One; produces tones from PWM
1 k Ω resistor	One; in series with the piezo buzzer
Momentary push button	One; local acknowledgement using an internal pull-up
Optional red, green and blue LEDs	Week 8 demo only; three LEDs and three 220 Ω resistors

Four-band resistor colours: 220 Ω = red-red-brown; 1 k Ω = brown-black-red; 10 k Ω = brown-black-orange. The tolerance band is often gold. Five-band parts differ; use the kit labels or a multimeter if uncertain. No external button resistor is needed.

Files supplied

File	What to use it for
src/main.cpp	Complete Week 9 alarm and MQTT communication
include/alarm_logic.h	State machine, timing, buzzer pattern and NTC equation
include/lab_config.h	Pins, threshold, thermistor constants and command word
include/secrets.h.example	Copy to secrets.h and enter your Wi-Fi and group label
src/week8_demo.cpp	Separate lecture-style RSSI and three-LED pre-lab demo
platformio.ini	Two selectable build environments and pinned libraries
tests/alarm_logic_test.cpp	Host tests for alarm logic and temperature conversion

Suggested order

1. Enter your Wi-Fi details and a unique group label as described on page 3.
2. Complete the Week 8 demo and screenshots on pages 6-7.
3. Disconnect power and replace the LED circuit with the alarm wiring on page 2.
4. Upload the Week 9 environment, then follow the demonstration on page 5.
5. Practise the lecture comparison on page 8. It is explicitly assessed.

Both firmware builds and the host logic tests can be checked without hardware. Actual readings, buzzer loudness, Wi-Fi connectivity and assessment screenshots must be verified on your own assembled ESP32.

Breadboard wiring for Week 9

Disconnect USB while wiring. Keep the ESP32 beside the breadboard and use jumper wires to its labelled pins; this leaves the breadboard holes accessible. Use GPIO numbers printed on the board, not physical header positions. All grounds connect together. Use 3V3, never VIN or 5V, for the thermistor divider.

Circuit to build

```
ESP32 3V3 ---- [10 kΩ fixed] ----+----- [10 kΩ NTC] ---- GND
                                |
                                GPIO34

ESP32 GPIO25 ---- [1 kΩ] ---- piezo (+)
                                piezo (-) ----- GND

ESP32 GPIO27 ---- push button ----- GND
                        normally open; INPUT_PULLUP in code
```

On a typical breadboard, a-e of one numbered row are connected; f-j form a separate connected group. The centre gap separates these groups. Power rails run lengthwise and may be split halfway along the board. Use one continuous rail section, or bridge a split after checking continuity.

Exact example positions

These positions assume an empty standard breadboard; the electrical connections matter more than the row numbers.

Part or jumper	Put it here
ESP32 3V3 jumper	Positive power rail
ESP32 GND jumper	Negative power rail
10 kΩ fixed resistor	One end in positive rail; other end in a10
Thermistor	One leg in b10; other leg in b15
ADC wire	c10 to ESP32 GPIO34
Thermistor ground wire	c15 to negative rail
Buzzer drive wire	ESP32 GPIO25 to a20
1 kΩ series resistor	a20 to a24
Piezo positive leg	b24
Piezo negative leg	A separate row, for example b27; extend with a jumper if needed
Piezo ground wire	The negative-leg row, for example c27, to negative rail
Push button	Across the centre gap; connect one switched contact to GPIO27 and the other to the negative rail

Do not put both resistor ends, both thermistor legs or both buzzer legs in the same connected row. The divider junction is row 10: resistor, thermistor and GPIO34 all meet there.

Button and buzzer checks

A four-leg button has two permanently connected pairs. Find them using continuity mode: choose contacts that are disconnected when released and connected only when pressed. Do not assume the orientation from the plastic shape. Connecting GPIO27 and GND to the same permanent pair makes the input permanently pressed.

Use the direct circuit above for a small passive piezo. Two pins do not distinguish piezo from a magnetic coil or an active buzzer. If the part is an electromagnetic buzzer, use its specified transistor driver and flyback diode; do not connect that coil directly to the GPIO. An active buzzer contains its own oscillator and will not demonstrate selectable PWM pitches correctly. The 1 kΩ series resistor can make a piezo quieter; confirm it is audible before assessment.

Configure and upload the alarm

Wi-Fi and group settings

Copy `include/secrets.h.example` to `include/secrets.h`. The latter is ignored by git. Enter your own settings, using a 2.4 GHz Wi-Fi network or a compatible phone hotspot. Ordinary SSID/password code will not log into a captive portal or enterprise Wi-Fi network.

```
#pragma once
#define LAB_WIFI_SSID "YourNetworkName"
#define LAB_WIFI_PASSWORD "YourNetworkPassword"
#define LAB_GROUP_ID "harlen-group01"
```

Replace `harlen-group01` with a unique group label and use that same replacement in every topic in this guide. Avoid spaces, `/`, `+` and `#`. With multiple ESP32s, give each a distinct label, such as `harlen-group01-node1`. The MQTT client ID also includes the board MAC address; a PC must use a different client ID.

Build and upload in PlatformIO

Open this project in VS Code with PlatformIO. Under Project Tasks, choose `upesy_wrover`, then Build and Upload. Open Monitor at 115200 baud. Alternatively, run these in a PlatformIO terminal:

```
pio run -e upesy_wrover
pio run -e upesy_wrover -t upload
pio device monitor -b 115200
```

If `pio` is not on your Mac shell `PATH`, use `~/platformio/penv/bin/pio` in its place. The existing board setting is `upesy_wrover`; confirm that it matches your board. Pin choices and these binaries are for the original ESP32, not automatically for ESP32-C3 or S3. The two environments compile different source files, so their `setup()` and `loop()` functions never conflict.

The project pins `espressif32 7.0.1`, `PubSubClient 2.8` and `ArduinoJson 7.2.0`. That PlatformIO platform uses Arduino-ESP32 2.x LEDC calls. Do not paste the buzzer calls into a 3.x Arduino-core project without adapting its LEDC API.

Temperature measurement and threshold

The ADC takes eight calibrated millivolt readings every 100 ms and averages them. GPIO34 is on ADC1 and can be used while Wi-Fi is active. For the divider on page 2:

```
Rntc = 10000 * Vnode / (3.3 - Vnode)      [volts and ohms]
Tkelvin = 1 / (1/298.15 + ln(Rntc/10000)/3950)
Tcelsius = Tkelvin - 273.15 + TEMP_OFFSET_C
```

At 25 C, a nominal 10 kΩ thermistor gives about 1.65 V at GPIO34. Warming an NTC lowers its resistance and the node voltage, so the calculated temperature rises. Verify the thermistor specification; edit `NTC_NOMINAL_OHMS` and `NTC_BETA` if needed. `SUPPLY_MV` can be set to your measured 3V3 rail voltage. `TEMP_OFFSET_C` provides a simple room-temperature adjustment after the wiring and thermistor constants are correct.

Start with `TRIGGER_C = 30.0` and `HYSTERESIS_C = 2.0` in `include/lab_config.h`. Watch the actual room and finger-warmed readings. Choose a trigger above room temperature but below the temperature you can reach with your fingers, then rebuild and upload. Finger warming may not reach body-core temperature. Never use a flame.

MQTT Explorer and alarm commands

Download MQTT Explorer from <https://mqtt-explorer.com/>. Create a connection using MQTT over TCP, host broker.emqx.io, port 1883, encryption TLS off. The supplied code uses the lecture's public demo username emqx and password public. Use matching entries in Explorer. These are demo strings, not private account credentials. EMQX currently documents this endpoint and port [1].

In Advanced settings, use a unique PC client ID such as harlen-group01-explorer. Remove broad # and \$SYS/# subscriptions and add `iot2026/harlen-group01/#` at QoS 0. Save, go back and connect. Never publish to a topic containing #; it is a subscription wildcard.

Topic design

All following Week 9 topics begin with `iot2026/harlen-group01/fire/`.

Suffix	ESP32 role and payload
temperature	Publishes a number in C, e.g. 31.25; null for an invalid reading
status	Publishes JSON: state, acknowledgement, sensor, threshold, identity and timing
command	Subscribes to JSON acknowledgement commands from Explorer
availability	Publishes retained online; broker publishes retained offline via Last Will after unexpected loss

The hierarchy separates course, group, application and message purpose. It avoids accidental group collisions and lets Explorer subscribe to one group's subtree. ESP32 and Explorer are both MQTT clients. The broker routes publications to subscribers; Explorer is not the broker. MQTT uses a client ID to identify a connection and a topic to route data; these are different things.

Temperature and status are retained, normally published each second. State and acknowledgement changes request an immediate status refresh. A late subscriber sees the latest retained state. Check increasing `uptime_ms` and recent activity to distinguish live data from a stale retained value. QoS 0 is used for measurements and commands; if a command is lost, check status and retry. Last Will uses QoS 1, and its offline notification is not instantaneous.

Remote acknowledgement

1. Warm the sensor until status shows WARNING or EMERGENCY.
2. Copy the exact `alarm_id` from the newest status message.
3. In Explorer's Publish panel, enter this exact topic:

```
iot2026/harlen-group01/fire/command
```

4. Select JSON, QoS 0 and retain OFF. Publish the following, replacing the example ID with the real one:

```
{"command":"ack","alarm_id":"COPY_CURRENT_ALARM_ID_HERE"}
```

5. Confirm the buzzer stops and status changes to acknowledged: true and `ack_source`: remote. The hot alarm state remains WARNING or EMERGENCY. Cooling to ≤ 28 C returns it to NORMAL and clears acknowledgement.

The ID contains a random boot identifier and an alarm episode number. It stays the same during WARNING to EMERGENCY. A command for a previous episode or previous boot is rejected, so an old retained command cannot silence a new alarm. Always turn retain off anyway. Acknowledgement during NORMAL does not pre-silence the next alarm. Keys and values are case-sensitive. Invalid JSON, missing fields and unknown commands are ignored and reported on Serial where applicable.

This public broker is shared and unencrypted. Use it only for demonstration data. Group labels and alarm IDs prevent mix-ups; they are not authentication.

Demonstrate and explain the alarm

State and timing rules

Condition	Result
NORMAL and temperature > 30 C	Enter WARNING; start a new 2-second timer
WARNING, no acknowledgement, elapsed >= 2000 ms	Enter EMERGENCY
WARNING or EMERGENCY and temperature <= 28 C	Return to NORMAL and clear acknowledgement
Temperature between 28 C and 30 C	Keep the current alarm state; avoids chatter
Button press or valid remote ack	Silence current alarm; suppress escalation if still WARNING
Invalid/out-of-range sensor reading	Extra SENSOR_FAULT state with fast pattern and sensor_ok false

WARNING uses 1800 Hz for 200 ms of every 1000 ms. EMERGENCY uses 2800 Hz for 125 ms of every 250 ms. LEDC hardware generates the tone; millis() determines when each tone is on or off. SENSOR_FAULT uses the fast pattern. Valid readings after a fault re-evaluate temperature, starting a fresh WARNING if hot.

Complete demonstration sequence

1. Leave the thermistor at room temperature. Confirm NORMAL, acknowledged false, a plausible temperature and a silent buzzer. Open the temperature graph and status in Explorer.
2. Warm the thermistor with your fingers. Show WARNING when temperature exceeds the trigger and listen for the slow pattern.
3. Do not acknowledge. Keep it warm for more than 2 seconds. Show EMERGENCY and the faster pattern remotely and locally.
4. Press and release the button. Confirm silence, acknowledged true and ack_source local. State remains EMERGENCY while hot.
5. Let the thermistor cool to the reset temperature. Confirm NORMAL and acknowledged false. Each new alarm needs a new press, not a continuously held button.
6. Warm again, wait for EMERGENCY, copy the new alarm_id and publish the remote command from page 4. Show silence and ack_source remote.
7. Cool and repeat once more; acknowledge during WARNING before 2 seconds. Keep it warm for at least 3 more seconds and show WARNING remains acknowledged without escalation.
8. Optionally turn off Wi-Fi briefly. Local sensing, the buzzer, escalation and button should still work. Restore Wi-Fi; the current status is republished after reconnection.

Information flow and responsiveness

Thermistor resistance changes -> divider voltage -> ADC1 readings -> beta equation -> local state machine -> LEDC buzzer pattern. The loop sends a snapshot through a FreeRTOS queue -> MQTT task serialises JSON -> PubSubClient publishes -> EMQX broker routes -> MQTT Explorer displays.

In the reverse direction, Explorer publishes command JSON -> broker routes to the ESP32 subscription -> client.loop() invokes callback() -> JSON is validated and queued -> the main loop checks the alarm_id -> acknowledge() silences the pattern -> new status travels back to Explorer.

Unsigned millis() subtraction drives all alarm timing and works across timer wraparound. The button has a 30 ms debounce. There is no delay(2000) for escalation. A 1 ms loop yield allows scheduling. MQTT's loop runs about every 10 ms when network calls are not blocked. PubSubClient is used only by the network task; reconnect retries occur about every 3 seconds and cannot hold up the alarm task. A one-element snapshot queue keeps the newest state during outages. Missed offline transitions are not stored as a history; reconnect publishes the current state.

Practise the Week 8 lecture demo

Use the separate week8_demo environment before the alarm assessment. It follows the supplied lecture's Wi-Fi RSSI publication and JSON LED control, so the proof directly resembles printed slides 32-37. It shares your secrets.h settings but uses a separate /demo/ topic subtree.

Wire the three LEDs

Disconnect USB. Disconnect the Week 9 buzzer and button from GPIO25 and GPIO27 before attaching LEDs; these pins are reused. Use one 220 Ω resistor per LED. A 1 k Ω resistor can substitute for a dimmer LED if needed. Do not share one resistor among three LEDs.

LED	Connection
Red	GPIO25 -> 220 Ω -> LED anode; LED cathode -> GND
Green	GPIO26 -> 220 Ω -> LED anode; LED cathode -> GND
Blue	GPIO27 -> 220 Ω -> LED anode; LED cathode -> GND

The longer leg is normally the anode (+). The shorter leg and flat rim normally indicate the cathode (-); inspect the part if its legs have been trimmed. The resistor may be on either side of its LED as long as it is in series. Place the two LED legs in different connected rows.

The lecture screenshot uses LED GPIO0, GPIO38 and GPIO39. This project substitutes GPIO25, 26 and 27 because the existing target is a classic ESP32 WROVER: GPIO34-39 are input-only, and GPIO0 is a boot-strapping pin. Do not copy those slide pin numbers blindly onto your board.

Upload and connect

```
pio run -e week8_demo
pio run -e week8_demo -t upload
pio device monitor -b 115200
```

Use the same Explorer connection from page 4 and the group subscription `iot2026/harlen-group01/#`. Wait for MQTT connected and subscribed in Serial. Expand the group's demo branch.

Topic under iot2026/harlen-group01/demo/	Direction and expected value
wifiMeta/rssi	ESP32 publishes Wi-Fi signal strength, usually a negative dBm number, every 2 seconds
msgIn/led_group	ESP32 subscribes; Explorer publishes all three LED fields
status	Added ESP32 feedback: group, MAC-derived device ID, uptime and LED output values

Publish this at QoS 0, retain OFF, to `iot2026/harlen-group01/demo/msgIn/led_group`:

```
{"led_red": "1", "led_green": "0", "led_blue": "1"}
```

Red and blue should turn on and green should turn off. Change all three values to "0" to turn them off. These are strings, including quotes, matching the lecture callback design. All three fields are required. Use proper JSON with quoted field names; the slides' screenshots contain abbreviated examples.

The returned status confirms the ESP32 processed the command, while looking at the actual LEDs confirms the electrical outputs. A copy of your own publication appearing in Explorer alone does not prove the ESP32 received it. After capturing evidence, disconnect power, restore the page 2 wiring and upload `upesy_wrover` for Week 9.

Capture the required pre lab proof

Prepare the evidence before your lab starts so you can show it in the first ten minutes. Capture your own live system; the guide and example payloads are not proof of a completed hardware chain.

Screenshot one shows ESP32 to PC

1. Run the Week 8 demo and connect Explorer to the same broker as the ESP32.
2. Expand `iot2026`, your personal group label, `demo` and `wifiMeta`.
3. Click `rsi`. Open the small graph/wave control and wait for several updates.
4. Keep the broker connection and your group topic visible in the screenshot.
5. Show the `demo/status` identity or a Serial window beside Explorer with your group label and ESP32 client ID. Keep Wi-Fi passwords out of view.
6. Save the image with a descriptive name, such as `W8_harlen-group01_rsi.png`.

This reproduces the identifying role of `kaisesp32` in printed slide 37, using your own label. A live series of RSSI updates and increasing uptime are stronger evidence than one cached value.

Screenshot two shows PC to ESP32

1. Select the exact `demo/msgIn/led_group` publish topic under your group.
2. Show the valid JSON payload and publish it with retain OFF.
3. Show the returned `demo/status` with matching LED values and your group/device identity, or Serial saying the outputs were updated.
4. If possible, include a photo of the corresponding physical LEDs. Save the command and response screenshot with your group label in its filename.

For the Week 9 demonstration, collect additional status screenshots of NORMAL, WARNING, EMERGENCY, local acknowledgement and remote acknowledgement. Include the temperature and group identity. WARNING lasts only around two seconds if unacknowledged; Explorer's history or a screen recording can help capture it.

Explain the pre lab chain in your own words

"My ESP32 and MQTT Explorer are two clients connected to `broker.emqx.io`. The ESP32 publishes its Wi-Fi RSSI under my group's `demo` topic, and Explorer subscribes to that subtree. Explorer publishes JSON to the LED command topic. The broker routes it to the ESP32 subscription, and the callback decodes the three string values and changes the GPIO outputs. The status publication confirms processing."

Common pre lab problems

Symptom	Check
No USB upload device	Use a USB data cable, correct serial port and board; close other serial monitors
Wi-Fi never connects	SSID/password, 2.4 GHz hotspot, signal, and absence of captive portal
Broker connects then disconnects repeatedly	PC and ESP32 must have different client IDs; public broker may be busy
No topics in Explorer	Exact broker/port/group match; custom subscription saved; firmware uploaded
Publish is visible but LEDs do not change	Command topic, quoted string values, all three keys, LED polarity and actual running environment
MQTT connection error	Serial <code>rc=-2</code> indicates connection failure; <code>-4</code> is timeout; <code>4/5</code> indicate credential/authorisation rejection [3]

If the public service or campus network prevents connection, try a permitted hotspot. As an alternative public broker, `test.mosquitto.org` supports port 1883 without authentication [5]. Set `MQTT_HOST` to that hostname and `MQTT_USER/MQTT_PASSWORD` to empty strings, use blank Explorer credentials, and rebuild. Both clients must move together. A public broker can fail; prepare and verify your evidence before arriving.

Explain the changes from the lecture

Do not omit this explanation: the task specifies a direct one-point deduction if you cannot clearly describe what changed from the Week 8 demo. The comparison below refers to printed slides 32-33, PDF pages 1-2, and Explorer setup on printed slides 34-37.

Lecture demo	This lab implementation
WiFiClient wrapped by PubSubClient; ArduinoJson callback	Retained exactly this library and communication structure
broker.emqx.io:1883; unique client ID using MAC	Same default broker; group plus MAC-derived client ID; boot/episode alarm ID added
Publish Wi-Fi RSSI on kaisesp32/wifiMeta/rssi	Alarm publishes temperature and JSON status under iot2026/group/fire; separate pre-lab demo still publishes RSSI
Subscribe to kaisesp32/msgIn/led_group	Alarm subscribes to group/fire/command and resubscribes after each connection
Read led_red, led_green and led_blue string values and set LEDs	Validate command and alarm_id strings; queue acknowledgement to the alarm loop
Wi-Fi waits and repeated reconnect loops with delays	Alarm starts independently; a dedicated network task handles connection attempts
delay(2000) between RSSI publications; delay(5000) on reconnect failure	millis-based sampling, 1-second telemetry, state updates, 2-second escalation and beep timing
Three digital LED outputs	Thermistor divider, GPIO34 ADC1 input, passive buzzer on GPIO25 and debounced button on GPIO27
Basic payload handling	Length-limited JSON parsing, required-field/type checks and old-alarm rejection
Basic demonstration publication	Retained current measurements/status, availability Last Will, threshold hysteresis and sensor fault reporting

The lecture-style pre-lab program also changes the LED pins for this board, uses your group prefix, validates all three fields, adds returned LED status, and schedules publications using millis(). Its connection attempts can still block briefly; the Week 9 program puts networking in its own task specifically to protect alarm timing.

Four individual assessment answers to practise

1. Topics and roles: "The ESP32 publishes temperature, status and availability and subscribes to command. Explorer subscribes to my group's subtree and publishes command. The broker routes messages. The group level prevents topic collisions, and separating commands from measurements makes message direction clear."
2. Temperature and threshold: "The NTC resistance falls when warmed. The divider and beta equation produce Celsius. Above TRIGGER_C starts WARNING. After two seconds without acknowledgement it becomes EMERGENCY. Cooling to TRIGGER_C minus HYSTERESIS_C resets it." At 29 C from NORMAL, a 30 C trigger does not alarm. Changing it to 28 C and re-uploading makes 29 C trigger WARNING, with reset at 26 C. Exactly 30 C does not trigger the default strict greater-than comparison. An already active alarm remains active at 29 C until it cools to 28 C.
3. Remote command: "client.loop() receives the MQTT packet and invokes callback(). The callback validates JSON and queues the command; the main loop checks the current alarm ID and acknowledges it." For the tutor's modification, change ACK_COMMAND from "ack" to "silence" in lab_config.h, rebuild/upload, copy the new boot's alarm_id and send command "silence". The old "ack" command should now be rejected.
4. Timing and local control: "Elapsed unsigned millis() values control escalation and the buzzer's on/off pattern. LEDC makes the waveform in hardware. The loop samples and checks the debounced button without a long delay. Network calls run in another task, exchanging only queued data with the loop." Demonstrate both acknowledgement paths and explain why the state remains hot but silent.

Troubleshooting and preparation for InfluxDB

Alarm checks before assessment

Symptom	Likely explanation and action
Temperature falls when warmed	Divider is reversed or sensor is not the assumed NTC; rebuild the page 2 circuit
Implausible room temperature	Check 10 kΩ fixed resistor, NTC rating/B value, common ground and SUPPLY_MV
SENSOR_FAULT and null temperature	ADC below 100 mV or above 3000 mV, or calculated temperature outside -10 to 80 C; inspect disconnected/shorted wiring
Always WARNING at startup	Room temperature is above the chosen trigger; select a suitable finger-demonstration threshold
Silent WARNING or EMERGENCY	Check acknowledged flag, buzzer connections/type and button input; allow cooling to reset
Button never works or is always pressed	Identify its two switched contact groups; avoid permanently joined legs
Remote command does nothing	Correct /fire/command topic, exact command word, current alarm_id and valid JSON; check Serial
Alarm stays active below 30 C	Intentional hysteresis; it resets only at ≤ 28 C with defaults
Status looks frozen	Check uptime and recent messages, Wi-Fi and broker; retained data can outlast a connection

A SENSOR_FAULT indicates a detected invalid reading, not a complete sensor self-test. In-range wiring errors or wrong thermistor constants can still produce plausible but incorrect temperatures. The ADC and thermistor are suitable for a classroom demonstration, not calibrated safety instrumentation.

The next lab's ESP32 to InfluxDB requirement

The attached PDF contains only the Week 8 MQTT demo, printed slides 32-37. It does not include the last page of the Week 9 slides. Consequently, the required InfluxDB version, hosting arrangement and exact marking evidence cannot be confirmed from this attachment. Follow that page for the assessed method.

A working MQTT Explorer screenshot proves MQTT connectivity, not that values reached InfluxDB. For the additional chain, plan to show actual stored temperature samples with timestamps and your group identity in an InfluxDB query or graph.

A practical optional path is ESP32 -> MQTT broker -> Telegraf MQTT consumer -> InfluxDB. Telegraf is another subscriber; it parses the /fire/status JSON and writes data to the database. The supplied MQTT payload already includes numeric temperature_c and group/device identity, so the ESP32 code does not need to change. Telegraf supports MQTT input and JSON parsing [6, 7].

If your course uses InfluxDB 2.x, the example in extras/telegraf-influxdb2.conf provides a starting point:

1. Use your course's InfluxDB 2.x instance, create/select an organisation and a bucket, and create a token allowed to write to that bucket. Install Telegraf using its official platform instructions.
2. Edit the example topic and Telegraf client_id to your own group. Set the InfluxDB URL, organisation and bucket, and supply the token using the INFLUX_TOKEN environment variable. Keep that token out of screenshots and git.
3. Run telegraf --config extras/telegraf-influxdb2.conf with the token in its environment. Run the ESP32 alarm with a working sensor; wait for several new messages.
4. In InfluxDB Data Explorer, query the selected bucket, measurement fire_alarm and field temperature_c over the last 15 minutes. Select your group tag. Warm the thermistor and confirm new stored values rise over time.
5. Capture the graph or query result showing your group and timestamps. Retained MQTT status can create an initial old sample with a new ingestion time, so demonstrate continuing new samples.

This example is for 2.x and uses Telegraf receipt time, not ESP32 uptime, as the database timestamp. It has not been run against your database. InfluxDB 1.x or 3.x and a lecture-required direct-HTTP method need different setup. Do not treat the example as verified completion of the missing Week 9 requirement.

References and verification record

Lecture reference

W8. MQTT 2026 (1).pdf, supplied by you: PDF pages 1-6, printed lecture slides 32-37. The firmware retains its WiFiClient/PubSubClient/ArduinoJson structure, broker choice, callback pattern, MAC-based connection identity and subscription renewal. The guide explains the code changes on page 8 and reproduces the practical Explorer flow on pages 4, 6 and 7.

Official technical references

[1] EMQX public broker endpoint and TCP port. Checked 1 October 2026.

<https://www.emqx.com/en/mqtt/public-mqtt5-broker>

[2] MQTT Explorer downloads, topic tree, graphing and custom subscriptions.

<https://mqtt-explorer.com/>

[3] PubSubClient API, including connect, publish, subscribe, loop, connection-state codes, buffer size and socket timeout.

<https://pubsubclient.knolleary.net/api>

[4] Espressif Arduino ADC API, including calibrated millivolt readings and attenuation.

<https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html>

[5] Mosquitto public test broker and supported connection methods.

<https://test.mosquitto.org/>

[6] Telegraf MQTT consumer plugin.

https://docs.influxdata.com/telegraf/v1/input-plugins/mqtt_consumer/

[7] Telegraf JSON input parser and InfluxDB 2.x collection guidance.

https://docs.influxdata.com/telegraf/v1/data_formats/input/json/

<https://docs.influxdata.com/influxdb/v2/write-data/no-code/>

Software checks supplied

The host tests cover strict threshold triggering, hysteresis, the 2-second escalation boundary, acknowledgement, rearming, buzzer on/off intervals, millis() wraparound, sensor fault recovery and thermistor conversion. Run them from the project directory on a Mac with the command below:

```
clang++ -std=c++11 -Wall -Wextra -Werror -Iinclude \
  tests/alarm_logic_test.cpp -o /tmp/w9-alarm-test
/tmp/w9-alarm-test
```

Build both ESP32 programs with:

```
pio run -e upesy_wrover -e week8_demo
```

The delivered software was build-checked for the existing board configuration and the host logic tests passed. No claim is made that your physical wiring, thermistor calibration, actual broker connection or InfluxDB chain has been tested. Complete the demonstrations on your board and save your own evidence.

Final bench checklist

- Your unique group label appears in the topics and screenshots.
- Thermistor readings are plausible and increase when warmed.
- NORMAL, WARNING and EMERGENCY are visible in Explorer.
- Local and remote acknowledgement both silence the buzzer.
- Cooling resets the alarm and permits another demonstration.
- You can explain the exact changes from the Week 8 slides.
- Your Week 8 proof is ready before the first ten minutes of class.
- You have checked the missing Week 9 slide for the next InfluxDB deadline.